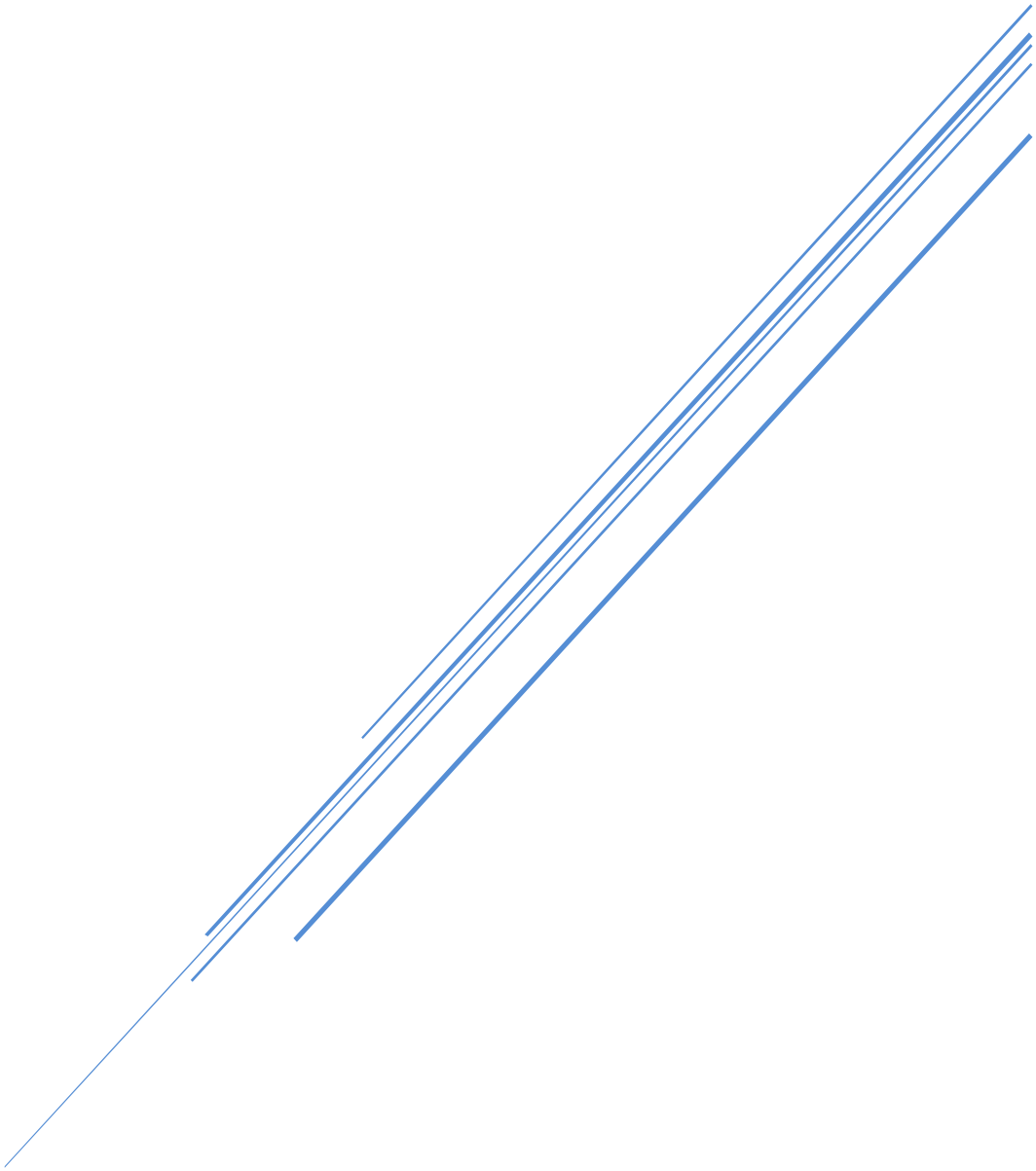
CHAINLAB: A C++ TERMINAL LOGIC ANIMATOR FOR BOTTOM-UP MATRIX CHAIN MULTIPLICATION

Design and Analysis of Algorithms course project report



Research Project
Design and analysis of algorithms

DESIGN AND ANALYSIS OF ALGORITHMS — COURSE PROJECT FINAL REPORT

COURSE CODE: BCSE204L

PROJECT AUTHOR: ANOMITRA PATRA (24BKT0096)

Created under the guidance and help of: Dr. Athira K.

ChainLab: A Frame-Based Logic Animator for Bottom-Up Matrix Chain Multiplication

Abstract—Matrix Chain Multiplication (MCM) is a classic dynamic-programming problem in which the order of parenthesization changes the number of scalar multiplications, even though matrix multiplication remains associative. A completed cost table can conceal the sequence of comparisons that produced its entries. This project presents ChainLab, a browser-based logic animator that turns the bottom-up MCM procedure into a navigable series of state frames. Each frame records the active matrix interval, candidate split, computed cost, relevant table state, comparison counters, and the pseudocode line to emphasize. The interface presents the minimum-cost table and split table together, supports forward and backward stepping, and animates reconstruction of the optimal parenthesization as a tree. A fixed left-to-right evaluation is included as a baseline for operation-count comparison. The implementation uses React and Vite, performs all calculation locally, and accepts synthetic dimension arrays without an external dataset. Unit tests cover input validation, a standard four-matrix example, a two-matrix chain, and the presence of the expected event types. This report describes the algorithmic model, interface architecture, implementation, verification scope, and limitations. The prototype is intended as an instructional artifact; formal evaluation with students remains future work.

Index Terms—Algorithm animation, dynamic programming, matrix chain multiplication, software visualization, stepthrough learning, traceback.

I. INTRODUCTION

Matrix Chain Multiplication asks how a sequence of compatible matrices should be parenthesized to minimize the number of scalar multiplications. Matrix multiplication is associative, so the parenthesized expressions produce the same mathematical product; nevertheless, intermediate dimensions change the amount of computation. The standard dynamic-programming algorithm computes the best cost for each interval and stores the split that achieves that cost [1].

The recurrence itself is compact, but following its execution requires coordinating the interval endpoints, chain length, candidate split, two smaller subproblem costs, and join multiplication cost. ChainLab focuses on making those intermediate decisions inspectable in a terminal without introducing a third-party graphics framework or a web runtime.

A. Problem Statement

The problem addressed is the gap between a static recurrence or completed DP table and the step-by-step reasoning used to fill it. The tool should expose the active interval and candidate values, show when a new best value replaces the prior value, record the chosen split, and allow the learner to revisit earlier frames. It should also present enough output to reconstruct the final parenthesization and understand the reported cost comparison.

B. Objectives

- Implement the standard bottom-up MCM recurrence in portable C++17 using a cost table m and a split table s.
- Represent base cases, interval initialization, candidate evaluation, minimum updates, interval finalization, and traceback as explicit frames.
- Provide terminal highlighting for the active cell and candidate subproblem cells, together with synchronized pseudocode emphasis.
- Support forward/backward stepping, autoplay, reset, jump-to-frame, playback-delay adjustment, and final-result navigation.
- Supply source code, a reproducible build, regression tests, and GitHub publishing instructions.

C. Significance and Scope

ChainLab is a learning aid that externalizes the intermediate state of a familiar algorithm; it does not introduce a new optimization recurrence. Research reviews of algorithm visualization report context-dependent results and caution against treating visualization alone as proof of educational effectiveness [2], [3]. For that reason, this report distinguishes implemented behavior and test evidence from claims about student learning. The program supports chains of 2 to 8 matrices so that the stored frames and terminal tables remain manageable.

II. LITERATURE REVIEW

The literature review is intentionally focused on the algorithmic source and established research about algorithm and program visualization. It is not presented as an exhaustive review of all MCM visualizers or as evidence that this implementation has already undergone a classroom evaluation.

A. Algorithmic Foundation

Cormen et al. present the classical dynamic-programming approach to MCM: define interval subproblems, evaluate every possible split, retain the minimum cost, and store enough information to reconstruct an optimal solution [1]. ChainLab follows this method directly. Displaying both m and s makes two related questions visible: how much work is required for a subchain, and which split explains that minimum.

B. Algorithm Visualization and Evaluation

Hundhausen, Douglas, and Stasko reviewed experimental studies of algorithm visualization and discussed how learning activity and evaluation methodology affect reported outcomes [2]. Their work provides a reason to avoid treating an attractive animation as an established learning intervention. Urquiza-Fuentes and Velázquez-Iturbide surveyed evaluations of program visualization and algorithm-animation systems, separating usability investigations from educational-effectiveness studies [3]. These reviews motivate ChainLab to be explicit about its controls and correctness tests while leaving instructional effectiveness as an open empirical question.

C. Design Need Addressed

The specific design goal is a coherent decision trace: a candidate cost should be associated with the relevant table values, the pseudocode line that describes the operation, the counter changes, and any update to the stored split. The frame log also makes backward inspection straightforward because it retains the earlier table state rather than attempting to reverse the recurrence. This is a concrete implementation objective, not a claim of uniqueness over existing educational tools.

III. SYSTEM ARCHITECTURE

ChainLab is a single-process terminal program. Input parsing and validation precede the algorithm engine. The engine calculates the DP tables and creates a complete frame sequence; the navigation controller selects the current frame, and the terminal renderer redraws the current tables, pseudocode, status message, and traceback state. No network service, database, or third-party library is required.

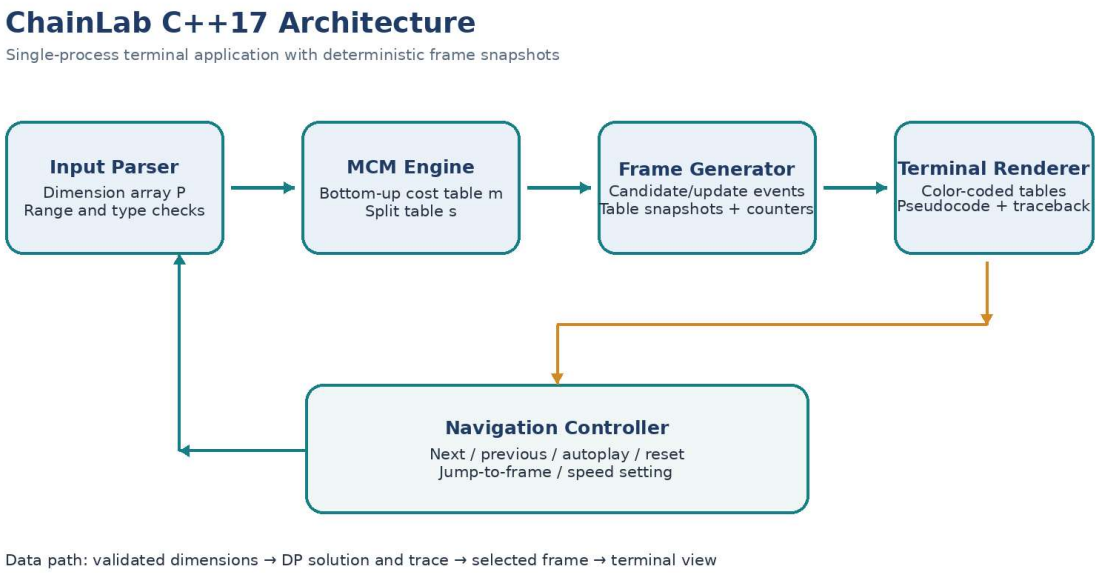


Fig. 1. C++ terminal architecture and the flow from validated input to a selected visualization frame.

A. Component Responsibilities

Component	Responsibility	Source module
Input parser	Reads positive integer dimensions and validates count and bounds.	src/main.cpp
MCM engine	Computes minimum costs and the split chosen for each interval.	src/mcm.cpp
Frame generator	Captures event type, interval, candidate values, counters, tables, and pseudocode line.	buildFrames() in src/mcm.cpp
Terminal renderer	Draws both DP tables, highlights relevant cells, prints code and status.	src/main.cpp
Navigation controller	Moves among frames, resets, jumps, and controls autoplay delay.	Command loop in src/main.cpp
Traceback builder	Creates the tree from stored split decisions and reveals nodes frame by frame.	src/mcm.cpp

B. Frame-Based State Model

Every frame contains a type, a user-readable message, the active interval and split (when applicable), candidate component costs, comparison/update counters, snapshots of m and s, and a pseudocode line identifier. Frames are constructed once at the start of the session. Moving backward therefore selects an earlier saved state; it does not require undoing algorithm operations.

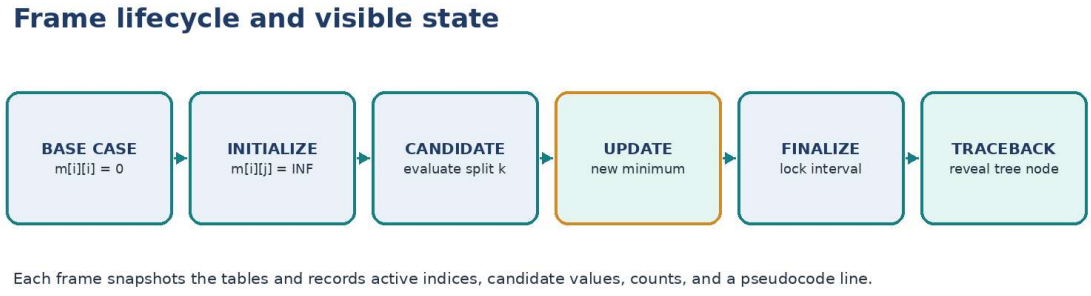


Fig. 2. Main event types used to make the DP computation and traceback observable.

Frame event	When emitted	Displayed purpose
Base case	Each diagonal entry m[i][i] becomes zero.	Shows that a one-matrix chain costs zero multiplications.
Initialize	A new interval is assigned an infinite best cost.	Identifies a subproblem whose candidates are about to be tested.
Candidate	Each possible split k is evaluated.	Shows left cost, right cost, join cost, total q, and previous best.
Update	A candidate improves the best known cost.	Records the improved cost and selected split.

Frame event	When emitted	Displayed purpose
Finalize	All split positions for an interval have been tried.	Marks the subproblem as complete.
Traceback	A tree node is visited after DP table completion.	Shows how the stored split table reconstructs the answer.

IV. MODULE DESCRIPTION

A. Input Processor

The dimension array is P = [p0, p1, ..., pn]. It defines n compatible matrices, where matrix Ai has dimensions p(i-1) by pi under one-based mathematical notation. The input accepts 3 to 9 positive integer values, corresponding to 2 to 8 matrices. Each dimension must be between 1 and 10,000. Invalid counts and out-of-range values produce an explanatory error before frame generation begins.

B. Core Logic Module

The engine initializes m[i][i] = 0. It then iterates over interval lengths from 2 through n; for each interval [i,j], every split k from i through j-1 is considered. The candidate value combines the best left and right subchains with the scalar multiplication cost of the final join. If the candidate is smaller than the current best, both the cost entry and its split are updated.

$$m[i,j] = \min \{ m[i,k] + m[k+1,j] + p[i-1] * p[k] * p[j] \},\ i <= k < j$$

(1) MCM recurrence in one-based mathematical notation; the C++ implementation uses zero-based vector indices.

C. Animation and Terminal Renderer

The renderer reads one frame and displays the event label, explanatory message, cost table, split table, statistics, and pseudocode. Cyan brackets mark the active table entry, while yellow braces identify the two subproblem values used in a candidate computation when ANSI coloring is enabled. The --no-color and --no-clear options provide a plain-output mode for terminals that handle ANSI control sequences poorly.

D. Traceback Module

After the DP tables are completed, the split table defines a binary decomposition of the full matrix chain. A leaf represents one matrix; an internal node represents an interval and the split selected for that interval. The tree is constructed in preorder and revealed progressively during traceback frames. The final parenthesization string is generated recursively from the same split table, so the tree and expression share one source of truth.

E. Interaction and Results

The command loop supports next, previous, autoplay, jump to an exact frame number, reset, change autoplay delay, jump to the final frame, help, and quit. At the final state, the terminal prints the optimal scalar multiplication count, the left-to-right baseline count, the percentage reduction in operation count, the optimal parenthesization, and the asymptotic complexity of the underlying dynamic program.

V. PROPOSED MODEL AND METHOD

A. Rationale for Bottom-Up Tabulation

Bottom-up tabulation produces a predictable order: single-matrix chains are resolved first, then intervals of length two, and so on until the full chain is solved. Each subproblem value needed for a longer interval is therefore already available. This order is well suited to an explicit state trace and allows the program to explain precisely which candidate split is being evaluated.

B. Frame Instrumentation

The core algorithm emits an immutable frame at each explanatory event. In a candidate frame, the active interval, split index, left and right subproblem costs, join cost, candidate total, previous best, counters, and table snapshots are stored. If the candidate improves the best value, an additional update frame is emitted after the table state has changed. At completion, traceback frames reveal the stored split decisions one node at a time.

$$q = m[i][k] + m[k+1][j] + p[i] * p[k+1] * p[j+1]$$

(2) Candidate cost in the zero-based indexing used internally by C++.

C. Complexity

For n matrices, the conventional dynamic-programming computation takes O(n^3) time and O(n^2) space for its cost and split tables [1]. ChainLab also stores a table snapshot in each frame. Since the trace can contain O(n^3) frames and each snapshot occupies O(n^2) space, the saved trace can require O(n^5) space in the worst case. The application therefore limits input to eight matrices to keep this teaching-oriented trade-off manageable.

VI. IMPLEMENTATION DETAILS

The application is implemented in standard C++17. The code uses vectors for the tables and frame log, long long values for scalar operation counts, exceptions for invalid input, and a command-driven terminal renderer. The optional CMake configuration builds the application and the regression-test executable. There are no third-party libraries or runtime services.

File / component	Technology	Purpose

File / component	Technology	Purpose
src/mcm.hpp	C++17 header	Defines frames, tree nodes, result structures, cost type, and algorithm interfaces.
src/mcm.cpp	C++17	Validation, bottom-up recurrence, candidate and update frames, parenthesization, and traceback.
src/main.cpp	C++17 standard library	Terminal display, highlighted tables, pseudocode, controls, input, and final summary.
tests/tests.cpp	C++17 assertions	Checks known results, parenthesization, event types, counters, and invalid dimensions.
CMakeLists.txt	CMake 3.16+	Builds the application and regression tests and registers a CTest test.
README.md	Markdown	Build commands, interface controls, algorithm explanation, limitations, and references.

A. Core Algorithm Sketch

```
for (int length = 2; length <= n; ++length) {
  for (int i = 0; i <= n - length; ++i) {
    int j = i + length - 1;
    m[i][j] = INF;
    for (int k = i; k < j; ++k) {
      q = m[i][k] + m[k+1][j] + p[i]*p[k+1]*p[j+1];
      if (q < m[i][j]) { m[i][j] = q; s[i][j] = k; }
    }
  }
}
```

The program creates a candidate frame for every k evaluation and an update frame whenever the minimum changes. It also records the finalization event for each interval and one traceback frame per tree node. The full source in the project archive is the authoritative implementation; the snippet above is a compact algorithmic summary.

VII. DATASET AND WORKED EXAMPLE

A. Input Description

The animator uses user-provided dimension arrays instead of an external dataset. The built-in example is P = [30, 35, 15, 5, 10], which describes four compatible matrices: A1 is 30 by 35, A2 is 35 by 15, A3 is 15 by 5, and A4 is 5 by 10. Since each matrix product is determined by the dimension array, no additional matrix values are required to count scalar multiplications.

B. Observed Output for the Example

Measure	Value
Number of matrices	4
Optimal scalar multiplication count	9,375
Left-to-right scalar multiplication count	19,500
Reduction in operation count	51.92%
One optimal parenthesization	((A1 * (A2 * A3)) * A4)

These values are regression-tested by the included C++ test program. The reduction is a comparison of theoretical scalar operation counts, not a measured speed-up in wall-clock time. Actual runtime depends on the numeric values, memory layout, compiler, and hardware; those factors are outside the current project scope.

C. Frame Sequence

The frame log begins with one base-case frame per matrix. It then logs each interval initialization, each candidate split evaluation, each improvement, and interval finalization. After the DP phase, the traceback phase reveals the decomposition tree in preorder. A user can inspect any saved frame using the jump command or move backward to compare the moment before and after an update.

VIII. VERIFICATION AND COMPARATIVE ANALYSIS

A. Regression Tests

The source bundle includes an assertion-based test executable. The test groups cover a two-matrix chain with a directly calculable cost, the four-matrix example above, the presence and ordering of event types, traceback node count, and rejection of invalid dimension values. In the preparation environment, the project compiled under C++17 with GCC 14.2.0; all five test groups passed through the registered CTest target.

Test group	Expected behavior	Result
Simple two-matrix chain	For dimensions [10,20,30], cost is 6,000 and parenthesization is (A1 * A2).	PASS
Four-matrix known example	Optimal cost 9,375 and left-to-right count 19,500.	PASS
Frame lifecycle	Base-case, candidate, update, finalize, and traceback events are present; counter is nondecreasing.	PASS
Traceback structure	Three matrices produce five tree nodes; stored splits form the reported expression.	PASS
Input validation	Zero dimensions and dimensions above 10,000 are rejected.	PASS

B. Comparison with a Static Explanation

Capability	Static recurrence/table	ChainLab C++
See intermediate cost changes	Usually requires a manually worked example	Saved candidate and update frames
Inspect prior state	Not available unless separately documented	Previous-frame navigation
See candidate components	May be shown in prose	Left/right/join costs displayed per candidate
Relate state to pseudocode	Requires separate reading	Current line emphasized with each frame
Reconstruct final solution	Must infer from split records	Traceback tree and parenthesization shown

This comparison summarizes the features provided by this project relative to a static explanation. It is not the result of a controlled benchmark against all existing visualization tools, and it does not claim that ChainLab is the first system to provide any listed feature.

C. Reproducible Build and Continuous Integration

The application and test target compiled with GCC 14.2.0 and CMake 3.31.6; CTest passed. The five assertion-based test groups cover known costs, frame ordering and counters, traceback structure, and invalid dimensions. The GitHub Actions workflow at .github/workflows/ci.yml repeats configuration, build, and test on pushes and pull requests. Run these commands from the repository root:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel
ctest --test-dir build --output-on-failure
```

D. Limitations

- The interface is terminal-based; it is not a graphical desktop application or an interactive website.
- Only chains of 2 to 8 matrices are accepted because each saved frame contains table snapshots.
- The baseline comparison uses fixed left-to-right parenthesization and theoretical scalar multiplication counts only.
- Autoplay can be interrupted through the terminal interrupt signal; the standard-library command loop does not provide raw-key asynchronous pause/resume.
- No controlled user study has been performed, so the report makes no claim about measured learning gains.
- GitHub repository publishing must be completed by the project owner; this preparation does not create a remote repository or a live browser demo.

IX. CONCLUSION AND FUTURE DEVELOPMENTS

ChainLab implements bottom-up Matrix Chain Multiplication as a frame-based C++17 terminal animator. The application combines the cost and split tables with candidate-cost explanations, code-line emphasis, counters, navigation controls, and a progressively revealed traceback tree. Known-example and validation tests provide a reproducible correctness check for the algorithmic core. The project prioritizes inspectability and ease of compilation over graphical rendering and avoids external runtime dependencies.

Future extensions could add a graphical interface using a C++ toolkit, export of frame traces as JSON or CSV, learner prediction prompts before each split, and additional algorithms that share the same frame abstraction. A formal usability or learning evaluation would be necessary before making stronger claims about educational impact. The most immediate next steps are to run the build on the target department environment, publish the repository, and record the repository URL below.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009, Chapter 15. Publisher: https://mitpress.mit.edu/9780262033848/introduction-to-algorithms.

[2] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, "A Meta-Study of Algorithm Visualization Effectiveness," *Journal of Visual Languages & Computing*, vol. 13, no. 3, pp. 259-290, 2002, doi: 10.1006/jvlc.2002.0237. https://www.sciencedirect.com/science/article/pii/S1045926X02902375.

[3] J. Urquiza-Fuentes and J. A. Velazquez-Iturbide, "A Survey of Successful Evaluations of Program Visualization and Algorithm Animation Systems," *ACM Transactions on Computing Education*, vol. 9, no. 2, Article 9, 2009, doi: 10.1145/1538234.1538236. https://doi.org/10.1145/1538234.1538236.

GITHUB REPOSITORY

Repository URL: https://github.com/quantum-hex577/mcm-logic-animator